

Flash v.2 Game Plan (Iterative & Modular Workflow)

Core Value Proposition & Product Vision

Flash is designed as a balanced, dual-sided ecosystem: it is a premier destination for clients to discover tattoo artists just as much as it is an automated booking and workflow suite for artists to get booked.

- **For Clients (Discovery & Inclusive Search):** A visual, style-driven marketplace to discover artists by location (local radius and guest spots) with dedicated community preference filters (e.g., Queer, Women, BIPOC, inclusive/safe spaces), browse available flash designs in real time, and book securely with transparent pricing and loyalty rewards.
- **For Artists (Booking & Client Acquisition):** A frictionless scheduling and CRM suite that eliminates manual admin, protects deposits, and continuously drives organic client acquisition through active marketplace discovery rather than relying solely on external social followings.

Overview & Tooling Pipeline

- **Platform Scope & Architecture Constraint (Strictly Native Mobile):** Flash is built 100% as a native cross-platform mobile application (iOS & Android via Flutter) for both clients and artists. No web-based application will be developed. Web presence is strictly restricted to a lightweight marketing landing page for product showcases, SEO, and directing users to App Store / Google Play downloads.
- **Framework & Language:** Flutter & Dart (Native cross-platform mobile client)
- **Design & Prototyping:** Figma + Google Stitch (Layouts, Design System Tokens, Screen Prototypes)
- **IDE & Agent Orchestration:** Google Antigravity (Manager View multi-agent parallel execution)
- **Models:** Gemini Flash (Rapid Flutter widget creation, styling, Dart design token translation) & Gemini Pro (Complex state management, architecture, Flutter Firebase SDK, security rules)
- **Backend:** Firebase (Authentication, Cloud Firestore, Firebase Storage, Security Rules)
- **Data Protection & Disaster Recovery:** Automated Cloud Firestore Managed Exports to

an isolated Cloud Storage backup bucket with Object Versioning and 30-day lifecycle retention.

Current Implementation Status & Feature Gap Analysis

Feature Domain	What's Implemented	What Still Needs to Be Built
Authentication & Core User Data	✅ Firebase Auth (Email, Google, Apple, Guest, Phone MFA), base UserProfile model & Firestore onboarding	🕒 Public artist profile schemas, portfolio image storage
Data Storage & Media	✅ Basic users & core_config collections in Firestore	🕒 Firebase Storage for tattoo flash / portfolios / reference photos, updated Security Rules for public reading
Booking & Appointments	❌ None	🕒 appointments & availability Firestore collections, booking workflow, device calendar export
Direct Messaging (Chat)	❌ None	🕒 Realtime conversations & messages Firestore streams, push notifications (FCM)
Payments & Deposits	🟡 CoreConfig fee placeholders	🕒 Stripe / Stripe Connect integration, Cloud Functions for PaymentIntents & webhooks
Server Logic & Automation	❌ No Cloud Functions initialized	🕒 Firebase Cloud Functions for payments, push alerts, and appointment reminders

Automated QA & Deployment: Jules (Asynchronous VM testing running flutter test, vulnerability scanning, PR generation)

Concurrent Design-to-Code Game Plan

Phase-by-Phase Game Plan

Phase 1: Architecture & Security Foundation (Modular Baseline)

Goal: Establish a clean Flutter & Dart project structure, secure foundation, Auth baseline, fundamental Firestore Security Rules, and an automated backup pipeline.

- **Models:** Gemini Pro
- **Antigravity Configuration:** Single thread focusing on core Flutter architecture, security, and data protection.
- **Agent Assignments:**
 - **Backend / Architecture Agent:**
 - Initialize the Flutter project structure with Dart null-safety and package dependencies (firebase_core, firebase_auth, cloud_firestore, firebase_storage, flutter_dotenv).
 - Setup Firebase Auth configuration (Apple & Google Sign-In bindings for Flutter).
 - Establish base Firestore schema and Dart models for User Profiles and Core Configs.
 - Write strict base Firestore Security Rules (request.auth.uid == userId) denying unauthorized access by default.
 - Configure environment variables (flutter_dotenv, .env, secrets management).
 - Configure automated daily Firestore Managed Exports to an isolated Cloud Storage bucket with Object Versioning and 30-day lifecycle rules to prevent unrecoverable data loss.
 - **Jules Deployment Milestone #1:**
 - *Trigger:* Initial Flutter project setup, Firebase config, Security Rules, and Backup pipeline completed.
 - *Jules Task:* Spin up VM environment, run static security analysis on environment configs, verify flutter analyze, lint security rules, verify backup bucket access controls, and submit PR #1: Baseline Flutter Architecture, Security Rules & Backup Pipeline.

Phase 2: Design-to-Code Pipeline & UI Scaffolding (Figma/Stitch Integration)

Goal: Translate Figma & Google Stitch design outputs into decoupled Flutter widgets and

screens using Dart.

- **Model:** Gemini Flash (for fast Dart code generation from visual specs and lower latency)
- **Antigravity Configuration:** Manager View with parallel agent threads.
- **Agent Assignments:**
 - **Design System Agent:** Extracts design tokens (colors, typography, spacing, component variants) from Figma/Stitch specs into a centralized Dart theme file (AppTheme, AppColors, AppTextStyles).
 - **Frontend Agent A:** Scaffolds decoupled Flutter screen widgets using design assets from Stitch/Figma and mock Dart data models.
 - **Frontend Agent B:** Builds reusable Flutter UI components (custom buttons, text fields, cards, loading indicators).
 - **QA Agent:** Generates Flutter widget unit test specs alongside UI creation.
- **Model Toggle Protocol:** Switch to Gemini Pro if encountering complex layout engine constraints, multi-screen navigation routing (go_router), or custom state bindings.

Phase 3: Feature-Sliced Integration & State Synchronization

Goal: Connect completed Flutter UI modules to Firebase Auth, Firestore, and Cloud Storage incrementally using Dart state management.

- **Model:** Gemini Pro
- **Antigravity Configuration:** Manager View with synchronized threads.
- **Agent Assignments:**
 - **Backend Agent:** Extends Firestore collections/schema for the specific feature slice (schedules, booking forms, chats); configures Firebase Storage buckets for uploaded flash artwork and profile media; implements Dart repository layer with serialization (json_serializable / freezed) and offline caching.
 - **Frontend Agent:** Binds reactive state management (e.g., Riverpod / Bloc / Provider) to Flutter UI components created in Phase 2 for that feature slice.

Core Implementation Requirement: Native Booking Engine in Firestore

Instead of paying for or embedding an external service (like Calendly or Cal.com), build lightweight scheduling logic directly into the existing Flutter and Firestore stack.

- **How It Works:**
 - **Artist Availability:** Each artist document has an availability subcollection or map (e.g., working days: Wednesday–Saturday; hours: 12:00 PM–8:00 PM; session buffer: 30 minutes).
 - **Slot Generation:** When a client looks at an artist's profile, the app queries the artist's schedule and cross-references existing documents in the bookings collection

for that date. Any conflicting times are blacked out.

- **Frontend UI:** Use Flutter packages like `table_calendar` or `syncfusion_flutter_calendar` to render clean date-pickers that match the Figma design system.
- **Why This Is Best for Flash Ink:**
 - **Seamless Stripe Integration:** The booking slot is locked in the exact same transaction where the Stripe Connect deposit is paid.
 - **Tattoo-Specific Logic:** Standard booking tools do not understand tattoo workflows (e.g., reserving a piece of flash so nobody else can claim it, or requiring full-day holds vs. 2-hour flash slots).
 - **Zero Extra Costs:** Eliminating monthly per-seat SaaS subscription fees eating into margins.
- **Jules Deployment Milestone #2 (Iterative per Feature Slice):**
 - *Trigger:* A feature module's Flutter UI and backend flow are connected (e.g., Auth Slice, Artist Booking Slice, Flash Artwork Gallery Slice).
 - *Jules Task:* Execute automated flutter test integration tests in the VM for that module, verify Firestore & Storage Security Rules against unauthorized access attempts, run dependency scans, and submit PR #2.x: Feature Module Integration.

Phase 4: Business Logic & Native Features

Goal: Implement core application logic, push notifications, local storage, and native platform integrations in Flutter/Dart per feature module.

- **Model:** Gemini Pro
- **Antigravity Configuration:** Parallel threads scoped strictly by feature module.
- **Agent Assignments:**
 - **Feature Agent 1:** Native integrations via Flutter plugins (camera, push notifications with `firebase_messaging`, local secure storage, deep links).
 - **Feature Agent 2:** Business logic in Dart, error handling, and retry mechanisms.
- **Model Toggle Protocol:** Switch to Gemini Flash when making minor UI tweaks, copy updates, or visual refinements matching updated Stitch/Figma screens.

Phase 5: Hardening, Automated Testing & Release Candidate

Goal: Comprehensive end-to-end testing, disaster recovery verification, security hardening, and release build validation.

- **Model:** Gemini Pro
- **Antigravity Configuration:** Single focused QA/Refactoring thread.
- **Agent Assignments:**

- **QA Agent:** Writes end-to-end (E2E) integration test scripts using `integration_test` in Flutter covering edge cases, bad inputs, and offline recovery. Conducts a Disaster Recovery & Restore dry-run test from the backup bucket.
- **Jules Deployment Milestone #3:**
 - *Trigger:* Full application feature completeness reached.
 - *Jules Task:* Run full flutter test and integration test suites in isolated VM, execute memory leak and vulnerability sweeps, verify build integrity, validate restore capability, and submit PR #3: Final Release Candidate Build.

Flash vs. Pro Decision Matrix

Task / Scenario	Recommended Model	Reason
Extracting design tokens & layout from Figma/Stitch	Gemini Flash	Fast layout and Flutter widget translation
Building static Flutter UI screens with mock data	Gemini Flash	High speed, low latency boilerplate creation
Minor styling tweaks or visual polish in Dart	Gemini Flash	Cost and time efficient
Base & Extended Firestore Security Rules	Gemini Pro	Requires deep security reasoning
Automated Backup & Disaster Recovery Architecture	Gemini Pro	Critical infrastructure & Cloud Storage policy setup
Flutter State Management & Reactive Data Flow	Gemini Pro	Handles multi-file context and complex logic
Firebase Flutter Plugins & Native SDK Bindings	Gemini Pro	Prevents subtle async/sync bugs

Task / Scenario	Recommended Model	Reason
Multi-file Dart refactoring or cross-agent debugging	Gemini Pro	Higher context reasoning capability

Antigravity Chat Window & Context Protocol

1. **Single Responsibility per Thread:** Scope threads tightly to a single UI component, feature slice, or backend service.
2. **The 3-Iteration Rule (Stop & Reset):** If an agent fails to resolve a bug or build error after 3 attempts, stop chatting in that thread to prevent context accumulation of failed attempts.
3. **Handshake Protocol for Bug Reset:**
 - o Ask the failing agent: *"Summarize the current problem, code changes attempted, exact error logs, and hypothesized root cause in a concise technical summary."*
 - o Close the thread, open a fresh thread with Gemini Pro, paste the summary, and request a clean approach.
4. **Git Branch & Feature-Slice Synchronization:** Link each Antigravity thread to a git feature branch. Only merge into main after Jules validates the feature slice PR in its VM. Always pull main before starting new threads.

Agent Prompt Library

Phase 1 Prompt: Architecture, Security & Backup Baseline (Gemini Pro)

Act as a Senior Flutter & Mobile Systems Architect. Set up the baseline architecture, Firebase security foundation, and automated data backup strategy for our Flutter application inside the current workspace.

Tasks:

1. Initialize the Flutter & Dart project structure with proper directory separation (lib/core, lib/features, lib/services) and configure environment secrets management (flutter_dotenv, .env) so no API keys or credentials are hardcoded.
2. Set up Firebase SDK initialization (firebase_core) and configure Firebase Auth bindings for Apple and Google Sign-In in Flutter.
3. Define the baseline Firestore schema and typed Dart models for the `users` collection (e.g., uid, email, displayName, createdAt).
4. Create a strict `firestore.rules` file that denies all read/write operations by default, allowing read/write only if `request.auth.uid == resource.data.userId` (or for authenticated user profile creation).
5. Configure the automated data protection pipeline: set up a scheduled daily Firestore Managed Export (via

Cloud Functions/Cloud Scheduler or export script) that sends database snapshots to an isolated Cloud Storage backup bucket with Object Versioning and 30-day lifecycle retention.

Constraints: Prioritize Dart null-safety, type safety, clean architecture, and security best practices. Do not build UI screens yet.

Phase 2 Prompt: Design System & UI Scaffolding (Gemini Flash)

Act as a Flutter UI Engineer. Convert the following design specifications from Figma/Google Stitch into Flutter widgets in Dart.

Inputs: [Paste design token values, color codes, typography styles, or layout specs here]

Tasks:

1. Create a centralized theme file (`lib/core/theme/app\_theme.dart`) containing extracted design tokens (colors, typography, spacing, border radii).
2. Scaffold the [Screen/Widget Name] UI using these theme tokens and mock Dart data models for display.
3. Ensure Flutter widgets are modular, decoupled from global state, and fully responsive across mobile screen sizes.

Constraints: Use strict layout styling matching the design tokens. Do not connect to real Firebase APIs yet—use mock Dart models only.

Phase 3 Prompt: Feature-Sliced Integration (Gemini Pro)

Act as a Full-Stack Flutter Engineer. Integrate the Flutter UI component [Screen/Widget Name] with the Firebase backend for the [Feature Name] module.

Tasks:

1. Create the repository layer and Firestore data mappers in Dart for [Feature Name] (e.g., artist schedules, custom flash designs, booking forms, or chat threads).
2. Configure Firebase Cloud Storage buckets and security rules if media uploads (e.g., artwork, attachments) are required.
3. Extend Firestore Security Rules to support required read/write operations for this feature while maintaining strict user authorization checks.
4. Bind the reactive state management (Riverpod/Bloc/Provider) to the Flutter UI component created in Phase 2.
5. Implement offline caching and graceful loading/error state handling.

Constraints: Ensure all database calls are authorized and handled asynchronously with proper error boundaries.

Phase 4 Prompt: Native Feature & Logic Integration (Gemini Pro)

Act as a Flutter Native Features Engineer. Implement native device capabilities for the [Feature Name] module in Dart.

Tasks:

1. Integrate native device APIs via Flutter plugins [e.g., Push Notifications with `firebase_messaging` / Camera / `flutter_secure_storage` / Deep Links].
2. Implement background task handling and error recovery mechanisms in Dart.
3. Ensure platform-specific permissions (iOS `Info.plist` / Android `AndroidManifest.xml`) are requested gracefully with appropriate user prompts and fallback states.

Constraints: Follow Flutter and native platform best practices for permissions and resource management.

Phase 5 Prompt: E2E Testing Suite & Disaster Recovery (Gemini Pro)

Act as a Lead Flutter QA Engineer. Build a comprehensive automated test suite for the [Feature Module Name] and verify disaster recovery readiness.

Tasks:

1. Write unit tests for Dart data mappers and state transitions.
2. Write Flutter integration tests (`'integration_test'`) covering happy paths, edge cases (e.g., network failure, bad inputs), and authentication expiration.
3. Include a disaster recovery verification test that confirms backup snapshots are being created and can be restored.
4. Ensure test scripts are structured to execute cleanly via `'flutter test'` in an automated CI/VM environment.

Constraints: Aim for high test coverage on critical business logic, security validation checks, and data protection.

Handshake Reset Prompt (3-Fail Protocol)

We are stopping this thread to reset context. Provide a concise technical summary of the current state:

1. The exact goal or task being attempted.
2. The code changes applied so far and files modified in the Flutter codebase.
3. The exact error messages or unexpected behavior observed.
4. The hypothesized root cause and what approaches failed.

Format this as a clean, structured summary that can be pasted directly into a fresh thread.

Jules Deployment Trigger Prompt (Async VM QA & PR)

Act as Automated QA Agent (Jules). Execute full VM testing and security validation for the target feature branch.

Tasks:

1. Spin up the Flutter test environment and execute all `'flutter test'`, widget, and security rule tests.
2. Perform static vulnerability scanning on Dart dependencies and environment configurations.
3. Verify that all Firestore Security Rules pass access control linting and backup bucket configurations are secure.

4. If all tests pass, submit a verified Pull Request titled: "[Milestone Name]: Verified Implementation, Security Rules & Test Suite".

Feature Ideas & Backlog Sandbox

A dedicated repository to capture, categorize, and prioritize new feature concepts, user flow improvements, and on-the-spot ideas before they are scheduled into specific development phases.

1. Artist Management & Studio Tools

Ideas for artist workflows, flash management, portfolio controls, and scheduling.

- **Artist Incentives ('The Studio Hustle'):**
 - **Fee-Free Milestones (Volume Discounts):** Waive or reduce Flash Ink's platform cut for the remainder of the month once an artist completes a target threshold of bookings (e.g., 'Hit 10 bookings this month, pay 0% platform fees on the rest').
 - **Artist-to-Artist Referral Bounties:** When an active artist refers a peer who completes their first 3 paid bookings, grant both artists a month of zero platform fees or elevated search discovery placement.
- **Artist Profile Social Integration (Instagram Handles & Deep Linking):**
 - Display verified Instagram handles on public artist profile pages and booking sheets.
 - Enable direct deep linking to external Instagram profiles for social proof, portfolio cross-referencing, and artist credibility.
- **Marketing & Artist Acquisition (Tattoo School Outreach):**
 - **Tattoo School Partnerships & Outreach:** Go to tattoo schools and market directly to new and up-and-coming artists, offering a turnkey platform and tools to hit the ground running with client acquisition, booking management, and deposit handling as they launch their careers.
- **Beta Testing & Early Artist Collaboration (Shawna Tester Promotion / Discounted Session Incentive):**
 - **Collaborative Beta Testing Discount:** Partner with resident artist Shawna to offer clients a designated percentage off their tattoo session in exchange for participating as beta testers.
 - **End-to-End Workflow Validation:** Recruits real tattoo clients to test the live scheduling flow, deposit checkout, appointment check-in, and review loops under real-world conditions while providing actionable feedback.
 - **Artist Onboarding Feedback:** Provides hands-on testing of the artist-side CRM,

calendar management, and payout mechanisms with an active, invested practitioner.

2. Client Experience & Discovery

Ideas for clients browsing flash art, discovering local artists, saving collections, and visual search.

- **New User Onboarding Incentive (First Two Bookings Fee-Free):**
 - **Two Free Bookings Promotion:** New users receive their first two bookings completely free of platform booking fees (0% Flash Ink fee on appointments #1 and #2).
 - **Conversion & Trial Acceleration:** Lowers the barrier to entry for first-time clients exploring the platform, driving higher immediate conversion from app download to initial booking.
 - **Architecture & Tracking:** Governed via a counter field on the client profile (e.g., `free_bookings_remaining: 2` in Firestore), verified server-side through Cloud Functions during Stripe PaymentIntent generation to prevent promotional abuse.
- **Gamified Client Referral Program & 'Ink Pass' Loyalty Engine:**
 - **Viral Referral Sharing Mechanics:** Unique client referral links/codes that reward both the referrer and the referred friend with 1 stamp on their 'Tattoo Stamp Card' upon the friend's first completed deposit.
 - **3-Booking Milestone Rewards ('Ink Pass'):**
 - **\$30 Flash Credit:** Platform-subsidized credit applied toward the 4th booking session.
 - **0% Platform Booking Fees:** Fee waiver on the client's next 2 appointments.
 - **VIP Flash Drop Access:** 15-minute priority early-access window to claim limited-edition flash sheets.
 - **In-App Visual Stamp Card:** Real-time visual progress tracker embedded in the client profile/Tattoo Passport displaying progress toward reward milestones (e.g., '2 of 3 Bookings Completed').
 - **Backend Architecture & Security:** Handled in Firebase Cloud Functions for referral code generation, fraud prevention/anti-abuse validation, and automated reward issuance.
- **Featured Local Artists & Spotlight Feed (Exploratory / Criteria TBD):**
 - **Core Concept:** Dedicated discovery feed spotlight and geo-targeted carousel highlighting local artists within the user's city/radius.
 - **Selection Criteria & Mechanics (Under Active Evaluation):** Framework for determining featured placement is TBD (exploring options such as algorithmic rotation based on booking velocity/top client reviews, weekly editorial spotlights, or boosted discovery granted via artist referral milestones).

3. Booking, Payments & Consultations

Ideas for appointment scheduling, deposit handling, intake forms, and chat messaging.

- **Native Booking Engine in Firestore (Approved Core Requirement):** Build native scheduling logic directly in Flutter and Firestore rather than embedding third-party booking tools (e.g., Calendly, Cal.com), enabling transactional Stripe deposit holds and tattoo-specific flash reservations. (See full technical specifications in Phase 3).
- **'Flash Pass' Premium Subscription Tier (Client & Collector Plan):**
 - **Zero Platform Booking Fees:** 100% waiver of platform transaction and booking protection fees across all appointments and deposit checkouts.
 - **Priority Early Access to Flash Drops:** 30-minute exclusive early claiming window for new flash sheets, guest spot bookings, and limited-edition designs before public drops.
 - **Monthly Tattoo Credits:** Monthly recurring platform credits (e.g., \$10/month) that roll over and can be applied toward deposits or final session balances with participating artists.
 - **Extended Cart Reservation Lock:** Extended 15-minute checkout hold (up from the standard 10 minutes) on one-off flash pieces to prevent high-volume drop race conditions.
 - **Premium Tattoo Passport & Archival Tools:** High-resolution digital artwork storage, priority direct messaging with artists, and custom curated flash wishlists.
 - **Technical & Billing Infrastructure:** Implemented via Stripe Billing / App Store Subscriptions with real-time entitlement validation synced through Firebase Cloud Functions and Firestore security rules.
- **Anti-Fraud Deposit Escrow & In-Person Check-In:**
 - **Deposit Escrow Hold:** Protect clients and artists against scammers by holding deposit funds in platform escrow rather than releasing them immediately.
 - **Mutual Appointment Check-In Verification:** Deposit funds are released/transferred to the artist only after the artist and client complete an in-person "check-in" (e.g., mutual in-app confirmation or QR code verification) when meeting for the session.

4. Flash Art & Media Innovations

Ideas for AR preview, stencil exports, high-res zoom, watermarking, and colorway variations.

- *(Drop new ideas here)*

5. Future Explorations & Backlog

Uncategorized ideas, moonshots, and future release considerations.

- *(Drop new ideas here)*

Brainstorming with Shawna

- **Touch-Up Booking Option:** Provide clients with the option to specifically select and schedule tattoo touch-up appointments during the booking flow.
- **Follow-Up & Multi-Session Bookings:** Add the ability for clients and artists to schedule follow-up appointments and manage multi-session projects seamlessly.